

PREPARED BY

 SHIVAM JHA

ROLE

SOFTWARE ENGINEER
INTERM

TECHNOLOGY STACK:

-  PYTHON
-  FASTAPI
-  PYDANTIC
-  UVICORN
-  SWAGGER / OPENAPI
-  GIT

PROJECT NEXELIS – INTERN-TO-ENGINEER BOOTCAMP DAY 12

FASTAPI EMPLOYEE MANAGEMENT API

TECHNICAL DOCUMENTATION



PROJECT NEXELIS – INTERNAL DOCUMENTATION

Table of Contents:-

- 1 Day 12 Objective
- 2 Introduction to FastAPI
- 3 FastAPI Project Structure
- 4 Routes
- 5 Path Parameters
- 6 Query Parameters
- 7 Request Models
- 8 Response Models
- 9 Pydantic Validation

10 Swagger / OpenAPI

11 Dependency Concepts

1. Day 12 Objective:-

Objective:-

To build the **first real backend API** using FastAPI. The API is an **Employee Management API** that can create, read, update, and delete employee records, with proper validation, status codes, error handling, and Swagger documentation.

What I Learned

- FastAPI project structure
- Routes
- Path parameters
- Query parameters
- Request models
- Response models
- Pydantic validation
- Swagger / OpenAPI
- Dependency concepts

Practical Task:-

Build an **Employee Management API** with these five endpoints:

Method	Endpoint	Purpose
POST	/employees	Create a new employee

GET	/employees	Get the list of all employees
GET	/employees/{id}	Get one employee by ID
PUT	/employees/{id}	Update an existing employee
DELETE	/employees/{id}	Delete an employee

Requirements

- Proper data validation
- Proper HTTP status codes
- Error handling
- Swagger documentation

Deliverables:-

Deliverable	Status
Git repository	Done
FastAPI application	Done
README file	Done
Swagger screenshot	Done
API documentation (this document)	Done

Real-Life Connection:-

Every company keeps employee data: names, emails, departments, and salaries. An HR system needs to add new joiners, view staff lists, update details, and remove people who leave. This is exactly what my API does, and it is the same idea behind real HR software.

The objective of Day 12 was to build my first real backend API, an Employee Management API in FastAPI with all CRUD operations, validation, error handling, and Swagger documentation.

2. Introduction to FastAPI

FastAPI is a modern, fast Python web framework used to build APIs. An API (Application Programming Interface) is a way for two software systems to talk to each other, for example a mobile app talking to a server.

Key Points

- FastAPI is written in Python and built on **Starlette** (web part) and **Pydantic** (data validation part).
- It is very fast, comparable to Node.js and Go in performance.
- It creates **automatic documentation** (Swagger UI) without extra work.
- It uses Python **type hints** (like `int`, `str`) to validate data automatically.
- It supports both normal (`def`) and asynchronous (`async def`) functions.
- It needs less code than many older frameworks, so there are fewer bugs.

Why it is used

- To build backend APIs quickly and with less code.
- To catch wrong or missing data automatically before it reaches the database.
- To give developers and testers ready-made, interactive API documentation.
- It is easy to learn, which makes it a good first backend framework.

Real-Life Example

Think of a **restaurant**:

- The **customer** is the mobile app or website (client).
- The **waiter** is the API (FastAPI).
- The **kitchen** is the database and backend logic.

The customer does not enter the kitchen. They tell the waiter what they want, the waiter carries the request to the kitchen, and then brings back the food (the response). FastAPI is the waiter between the user and the data.

FastAPI is a fast Python framework that helps me build APIs with automatic validation and automatic Swagger documentation.

3. FastAPI Project Structure

Project structure is the way we arrange our files and folders so that the code stays clean, organized, and easy to understand. Instead of writing everything in one file, we split the code into small files, and each file has one clear job.

Key Points

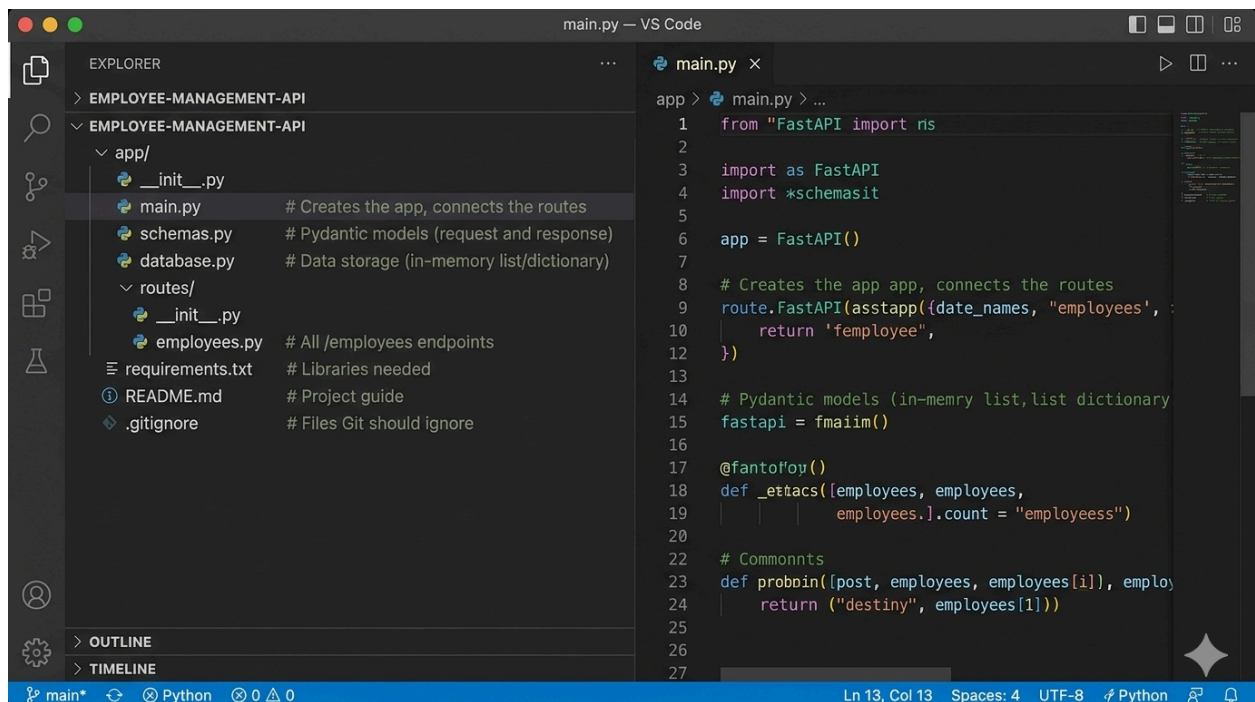
- A good structure separates **routes**, **data models (schemas)**, and **application setup**.
- Small files are easier to read, test, and fix.
- FastAPI does not force one structure, but a clear one is a best practice.
- Every project should have a `requirements.txt` (list of libraries) and a `README.md` (how to run it).
- A `.gitignore` file keeps unwanted files (like `venv/`) out of Git.

Why it is used

- To keep the code **clean and easy to maintain** as the project grows.
- To help teammates find things quickly.
- To make it easy to add new features without breaking old code.
- To follow the same professional standard used in real companies.

How it is used

Folder layout of my Employee Management API



What each file does:

File	Job
<code>main.py</code>	Starting point. Creates the FastAPI app and includes the routers.
<code>schemas.py</code>	Defines the shape and rules of the data (Pydantic models).
<code>database.py</code>	Stores the employee data.
<code>routes/employees.py</code>	Contains the five employee endpoints (POST, GET, PUT, DELETE).
<code>requirements.txt</code>	Lists libraries such as <code>fastapi</code> and <code>uvicorn</code> .
<code>README.md</code>	Explains what the project is and how to run it.

Think of a school building:

- The **main gate and office** is `main.py`, the entry point that controls everything.
- Each **classroom** is a separate file, such as `employees.py`, handling one subject only.
- The **rulebook** is `schemas.py`, which decides what is allowed.
- The **record room** is `database.py`, which stores the data.

If all subjects were taught in one giant room, it would be chaos. Splitting into rooms keeps everything organized.

I organized my project into separate files for the app, routes, and schemas, so the code stays clean and easy to maintain.

4. Routes:-

A route is a URL path that is connected to a Python function. When a client sends a request to that path using a specific HTTP method, FastAPI runs the matching function and returns the response.

Route = HTTP method + URL path + function

Key Points

- Routes are created using **decorators** such as `@app.get()`, `@app.post()`, `@app.put()`, and `@app.delete()`.
- The decorator tells FastAPI **which method** and **which URL** the function handles.
- The same URL can be used with different methods for different actions (for example, GET /employees and POST /employees).
- Routes can be grouped using an **APIRouter**, so each feature has its own file.
- The value returned by the function is automatically converted to **JSON**.

Types (HTTP Methods)

Method	Purpose	CRUD Action	Example
GET	Read data	Read	GET /employees
POST	Send new data	Create	POST /employees
PUT	Update existing data	Update	PUT /employees/1
DELETE	Remove data	Delete	DELETE /employees/1

Why it is used

- To decide **what action** the server takes for each request.
- To keep the API organized, with one clear address for each operation.
- To follow the standard **REST** style that all developers understand.

How it is used:-

Simple routes in one file:

python

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
@app.get("/employees")
```

```
def get_employees():
```

```
    return {"message": "List of all employees"}
```

```
@app.post("/employees")
```

```
def create_employee():
```

```
    return {"message": "Employee created"}
```

```
}
```

What each part means:

- `@app.get("/employees")` means: when a GET request comes to `/employees`, run the function below.
- `APIRouter` creates a group of related routes.
- `prefix="/employees"` adds `/employees` in front of every route in that group.
- `tags=["Employees"]` groups these routes together in Swagger.

A route connects an HTTP method and a URL to a Python function, so FastAPI knows what to do for each request.

5. Path Parameters

A path parameter is a variable part of the URL that is used to identify one specific resource. It is written inside curly braces `{}`, in the route and is passed to the function as an argument.

Example: in `/employees/5`, the number **5** is the path parameter (the employee ID).

Key Points

- Path parameters are written as `{name}` inside the URL path.
- The function must have an argument with **the same name**.
- FastAPI uses the **type hint** (`int`, `str`) to validate and convert the value automatically.
- Path parameters are **required**. The URL will not work without them.
- If the type is wrong (for example `/employees/abc` when `int` is expected), FastAPI automatically returns a **422 error**.
- They are mostly used to get, update, or delete **one specific item**.

Why it is used

- To point to **one exact record** (for example, one employee).
- To create clean, readable URLs that follow REST standards.
- To avoid sending the ID in the request body when it is not needed.

How it is used:-

Basic example:

python

```
@router.get("/{id}")
```

```
def get_employee(id: int):
```

```
    return {"employee_id": id}
```

How it works:

Request	What happens
GET /employees/1	<code>id = 1</code> , returns the employee with ID 1
GET /employees/99	Employee does not exist, returns 404 Not Found
GET /employees/abc	Not a number, returns 422 Validation Error

6. Query Parameters

A query parameter is an optional value added at the end of a URL after a question mark `?`. It is used to filter, search, sort, or limit the data that comes back.

Example: in `/employees?department=IT`, the part `department=IT` is the query parameter.

Key Points

- Query parameters come after **?** in the URL.
- Multiple parameters are joined with **&**, for example `/employees?department=IT&limit=5`.
- In FastAPI, any function argument that is **not** in the path is treated as a query parameter.
- They are usually **optional**, and you can give them **default values**.
- FastAPI uses **type hints** to validate them automatically (a wrong type gives a **422 error**).
- They are mostly used with **GET** requests.

Path Parameter vs Query Parameter

Point	Path Parameter	Query Parameter
Position in URL	Inside the path	After ?
Example	<code>/employees/5</code>	<code>/employees?department=IT</code>
Required or optional	Required	Usually optional
Purpose	Identify one item	Filter or search many items

Why it is used

- To filter data without creating a new route for every case.
- To support **searching**, **sorting**, and **pagination** (showing data page by page).
- To keep the API flexible: the same route works with or without filters.

Different Practical Examples:-

1. Search

GET `/api/employees?search=Shivam`

→ **Find employees matching Shivam.**

2. Sorting

GET `/api/employees?sort=joining_date`

→ *Return employees sorted by joining date.*

3. Pagination

GET /api/employees?page=2&limit=5

→ *Return the second page with 5 employees.*

4. Filtering

GET /api/employees?status=ACTIVE

→ *Return only active employees.*

7. Request Models

A request model is a Python class that describes the **shape and rules of the data a client sends** to the API, usually in the body of a POST or PUT request. In FastAPI, request models are created using Pydantic's `BaseModel`.

Key Points

- A request model is a class that inherits from `BaseModel`.
- Each field has a **name** and a **type** (for example `name: str`, `salary: float`).
- The data is sent as **JSON** in the request body.
- FastAPI reads the JSON, checks it against the model, and passes a ready-to-use object to the function.
- If the data is wrong or missing, FastAPI automatically returns a **422 error** with a clear message.
- Request models are used with **POST** (create) and **PUT** (update).
- GET and DELETE normally do not need a request body.

Why it is used

- To make sure the client sends **correct and complete data**.
- To avoid writing manual `if` checks for every field.
- To show the exact input format in **Swagger** automatically.
- To keep the code clean by defining the data shape in one place.

How it is used

Step 1: Create the model in `schemas.py`

```
python
from pydantic import BaseModel
class EmployeeCreate(BaseModel):
    name: str
    email: str
    department: str
    salary: float
```

Step 2: Use the model in the route

```
python
@router.post("/", status_code=201)
def create_employee(employee: EmployeeCreate):
    new_employee = employee.model_dump()
    new_employee["id"] = len(employees) + 1
    employees.append(new_employee)
    return new_employee
```

Example JSON sent by the client:

```
json
{
  "name": "Rahul Sharma",
  "email": "rahul@example.com",
  "department": "IT",
  "salary": 50000
}
```

Real-Life Example

Think of a **job application form**:

- The form has fixed fields: Name, Email, Department, Salary expected.

- If you leave a field empty or write text where a number is needed, the receptionist rejects the form.

The request model is that form. Only correctly filled data is accepted and passed to the office (the function).

A request model is a Pydantic class that defines what data the client must send, and FastAPI validates it automatically.

8. Response Models

A response model is a Python class that describes the **shape of the data the API sends back** to the client. In FastAPI, it is created with Pydantic's `BaseModel` and attached to a route using the `response_model` parameter.

Key Points

- A response model controls **what the client receives**, just as a request model controls what the client sends.
- It is set in the route decorator: `@router.get("/", response_model=EmployeeResponse)`.
- FastAPI **filters** the output so that only the fields in the model are returned.
- It can **hide sensitive fields** (for example a password or internal notes).
- It **validates** the output, so the API never sends data in the wrong format.
- It appears in **Swagger** as the example response, so users know what to expect.
- For a list of items, use `List[EmployeeResponse]`.

Request Model vs Response Model

Point	Request Model	Response Model
Direction	Client → API	API → Client
Used with	POST, PUT	All routes

Purpose	Validate incoming data	Control and validate outgoing data
Example	<code>EmployeeCreate</code> (no ID)	<code>EmployeeResponse</code> (with ID)

Why it is used

- To return data in a **clean and consistent** format.
- To **protect private data** by sending only the allowed fields.
- To **auto-generate** correct response documentation in Swagger.
- To let the server add fields the client did not send, such as the `id`.

How it is used

Step 1: Create the model in `schemas.py`

python

```
from pydantic import BaseModel
```

```
class EmployeeResponse(BaseModel):
```

```
    id: int
```

```
    name: str
```

```
    email: str
```

```
    department: str
```

```
    salary: float
```

Example response from the API:

json:-

```
{  
  
  "id": 1,  
  
  "name": "Rahul Sharma",  
  
  "email": "rahul@example.com",  
  
  "department": "IT",  
  
  "salary": 50000  
  
}
```

What each part means:

- `EmployeeResponse` defines what the client will see.
- `response_model=...` tells FastAPI to filter and validate the output using that class.
- `List[...]` is used when the route returns many items.

Real-Life Example

Think of a **bank statement**:

- The bank stores a lot of data about you: PIN, internal risk score, and staff notes.
- The statement that reaches you shows only the safe fields: name, account number, and balance.

The response model is that statement format. Everything private stays inside, and only approved information goes out.

A response model is a Pydantic class that decides exactly which fields the API sends back, so the output is clean, safe, and well documented.

9. Pydantic Validation

Pydantic is a Python library that **checks and cleans data** using type hints. Pydantic validation means making sure the data coming into the API has the correct type, format, and value range before the code uses it.

Key Points

- Pydantic is the data validation part of FastAPI.
- Validation is done by defining a class that inherits from `BaseModel`.
- **Type checking:** `name: str, salary: float, age: int`.
- **Extra rules** are added with `Field()`, for example minimum length, maximum length, and greater than or less than.
- **Email format** is checked with `EmailStr` (needs `pip install "pydantic[email]"`).
- **Optional fields** use `Optional` with a default value of `None`.
- Pydantic tries to **convert** data when possible (for example, the string `"5000"` becomes the number `5000`).
- If validation fails, FastAPI returns **422 Unprocessable Entity** with details of what went wrong.

Common Validation Rules

Rule	Meaning	Example
<code>min_length</code>	Minimum characters	<code>Field(min_length=2)</code>
<code>max_length</code>	Maximum characters	<code>Field(max_length=50)</code>

gt	Greater than	Field(gt=0)
ge	Greater than or equal to	Field(ge=18)
lt	Less than	Field(lt=100)
le	Less than or equal to	Field(le=65)
EmailStr	Must be a valid email	email: EmailStr

Why it is used

- To **stop bad data** before it reaches the database.
- To avoid writing many manual **if** checks.
- To give the client **clear error messages** about what is wrong.
- To keep the API safe and reliable.

How it works:

Input

Result

Valid name, email, salary	Data accepted
------------------------------	------------------

name has 1 character	422 error
----------------------	--------------

Email without @	422 error
-----------------	--------------

salary is 0 or negative	422 error
----------------------------	--------------

A required field is missing	422 error
--------------------------------	--------------

Real-Life Example

Think of an **airport security check**:

- Before you enter, your ticket, ID, and bag are checked.
- If anything is wrong, you are stopped at the gate and told the reason.
- Only correct passengers go inside.

Pydantic is that security gate. It checks every piece of data at the entrance, so wrong data never enters your application.

Pydantic validation automatically checks the type, format, and range of incoming data, and returns a clear 422 error when the data is wrong.

10. Swagger / OpenAPI

OpenAPI is a standard format for describing an API: its routes, inputs, outputs, and errors. **Swagger UI** is a web page that reads this description and shows it as interactive documentation, where you can test every endpoint from the browser. FastAPI creates both **automatically**.

Key Points

- FastAPI generates the OpenAPI description from your code (routes, models, type hints), so no extra work is needed.
- The documentation always stays in sync with the code.
- You can **test endpoints directly** in the browser using the "Try it out" button.
- It shows the request format, response format, and status codes for every route.

Types (Documentation Pages in FastAPI)

URL	What it is
<code>/docs</code>	Swagger UI : interactive, lets you test endpoints
<code>/redoc</code>	ReDoc : clean, read-only documentation
<code>/openapi.json</code>	The raw OpenAPI file (JSON) that both pages use

Why it is used

- To give developers, testers, and mentors **ready-made documentation**.

- To **test the API** without Postman or extra tools.
- To let frontend developers see exactly how to call the API.
- To save the time of writing documentation by hand.

Real-Life Example

Think of a **product user manual with a demo counter**:

- The manual tells you what each button does (the documentation).
- The demo counter lets you press the buttons and see what happens (Try it out).

Swagger is both: it explains the API and lets you test it at the same time.

Swagger UI is the interactive documentation that FastAPI creates automatically at /docs, where I can read and test all my endpoints.

11. Dependency Concepts

A **dependency** is a reusable piece of code (usually a function) that a route needs before it can run. **Dependency Injection** means FastAPI runs that function for you and gives the result to your route, using `Depends()`.

Key Points

- Dependencies are declared with `Depends()` in the function parameters.
- FastAPI **calls the dependency first**, then passes the result into the route.
- One dependency can be reused in **many routes**.
- Dependencies can use **path parameters, query parameters, and headers** just like routes.
- A dependency can **stop the request** by raising an error (for example, 401 Unauthorized).
- Dependencies are shown in Swagger automatically.

Types (Common Ways to Use Dependencies)

Type	Meaning	Example use
Function dependency	A simple function	Pagination values, API key check
Class dependency	A class used as a dependency	Grouping common parameters
Sub-dependency	A dependency that uses another dependency	Get user, then check role
Route-level or global dependency	Applied to a route or the whole app	Authentication for all routes

Why it is used

- To **avoid repeating** the same code in many routes (DRY: Don't Repeat Yourself).
- To share things like **database connections** and **login checks**.
- To keep routes short and clean.
- To make code easier to **test and change**.

How it works:

Step	What happens
1	Client sends a request to the route
2	FastAPI runs the dependency function first
3	If it passes, the result goes into the route
4	If it fails, the request stops with an error

Real-Life Example

Think of an **office entrance gate**:

- Before anyone reaches any department, the **security guard** checks the ID card.
- The same guard serves every department, so no department needs its own check.
- If the ID is wrong, the person is stopped at the gate.

The dependency is the security guard. It does a shared job before the route runs, and the routes only focus on their own work.

A dependency is reusable code that FastAPI runs before my route using Depends(), so I do not repeat the same logic in every endpoint.

12.API Documentation: Employee Management API

1. API Overview

The Employee Management API is a REST API developed using FastAPI to manage employee records through CRUD (Create, Read, Update, Delete) operations. It includes data validation, proper HTTP status codes, error handling, and automatic Swagger documentation.

2. Base URL

Interactive documentation (Swagger UI):

<http://127.0.0.1:8000/docs>

3. API Endpoints

Method	Endpoint	Description
POST	/employees	Create a new employee
GET	/employees	Retrieve all employees

GET	/employees/{emp_id}	Retrieve an employee by ID
PUT	/employees/{emp_id}	Update an employee
DELETE	/employees/{emp_id}	Delete an employee

4. Employee Data Structure

An employee record contains the following fields:

Field	Type	Description
id	Integer	Unique employee ID generated by the application
name	String	Employee name
email	String	Employee email address
department	String	Employee department
designation	String	Employee designation
salary	Float	Employee salary
is_active	Boolean	Employee active status

Example Employee

```
json
{
  "id": 1,
  "name": "Ananya Sharma",
  "email": "ananya.sharma@example.com",
```



```
"department": "Finance",  
"designation": "Accountant",  
"salary": 45000,  
"is_active": true  
}
```

5. POST /employees

Creates a new employee. The employee ID is automatically generated by the application.

Request Body

```
json  
{  
  "name": "Ananya Sharma",  
  "email": "ananya.sharma@example.com",  
  "department": "Finance",  
  "designation": "Accountant",  
  "salary": 45000,  
  "is_active": true  
}
```

Successful Response

Status Code: **201 Created**

```
json  
{  
  "id": 1,  
  "name": "Ananya Sharma",  
  "email": "ananya.sharma@example.com",  
  "department": "Finance",  
  "designation": "Accountant",  
  "salary": 45000,  
  "is_active": true  
}
```

Validation

The following validation rules are applied:

- `name` must contain between 2 and 50 characters.
- `email` must be a valid email address.
- `department` must contain between 2 and 50 characters.
- `designation` must contain between 2 and 50 characters.
- `salary` must be greater than 0.
- `is_active` must be a Boolean value.

Invalid request data results in a **422 Unprocessable Entity** response.

Example Validation Error

Request body:

```
json
{
  "name": "A",
  "email": "ananya-at-example",
  "department": "Finance",
  "designation": "Accountant",
  "salary": -1000,
  "is_active": true
}
```

Response (shortened), Status Code: **422 Unprocessable Entity**

```
json
{
  "detail": [
    {
      "loc": ["body", "name"],
      "msg": "String should have at least 2 characters"
    },
    {
```

```
    "loc": ["body", "email"],
    "msg": "value is not a valid email address"
  },
  {
    "loc": ["body", "salary"],
    "msg": "Input should be greater than 0"
  }
]
```

6. GET /employees

Retrieves all employees currently stored in the application.

Request

GET /employees

Successful Response

Status Code: **200 OK**

```
json
[
  {
    "id": 1,
    "name": "Ananya Sharma",
    "email": "ananya.sharma@example.com",
    "department": "Finance",
    "designation": "Accountant",
    "salary": 45000,
    "is_active": true
  },
  {
    "id": 2,
    "name": "Karan Mehta",
    "email": "karan.mehta@example.com",
```

```
"department": "Marketing",  
"designation": "Marketing Executive",  
"salary": 38000,  
"is_active": true  
}  
]
```

If no employees exist, the API returns an empty list with status code **200 OK**.

7. GET /employees/{emp_id}

Retrieves a specific employee using the employee ID.

Path Parameter

Parameter	Type	Description
emp_id	Integer	ID of the employee to retrieve

Example Request

GET /employees/2

Successful Response

Status Code: **200 OK**

```
json  
{  
  "id": 2,  
  "name": "Karan Mehta",  
  "email": "karan.mehta@example.com",  
  "department": "Marketing",  
  "designation": "Marketing Executive",  
  "salary": 38000,  
  "is_active": true  
}
```

```
}
```

Employee Not Found

Status Code: **404 Not Found**

```
json
```

```
{
```

```
  "detail": "Employee not found"
```

```
}
```

Invalid Employee ID

If the ID provided in the URL is not an integer (for example, [/employees/abc](#)), FastAPI returns a validation error.

Status Code: **422 Unprocessable Entity**

8. PUT /employees/{emp_id}

Updates an existing employee using the employee ID.

Path Parameter

Parameter	Type	Description
emp_id	Integer	ID of the employee to update

Example Request

PUT /employees/2

Request Body

```
json
```

```
{
```

```
  "name": "Karan Mehta",
```

```
  "email": "karan.mehta@example.com",
```

```
"department": "Marketing",  
"designation": "Senior Marketing Executive",  
"salary": 52000,  
"is_active": true  
}
```

Successful Response

Status Code: **200 OK**

```
json  
{  
  "id": 2,  
  "name": "Karan Mehta",  
  "email": "karan.mehta@example.com",  
  "department": "Marketing",  
  "designation": "Senior Marketing Executive",  
  "salary": 52000,  
  "is_active": true  
}
```

Employee Not Found

Status Code: **404 Not Found**

```
json  
{  
  "detail": "Employee not found"  
}
```

Validation Error

Invalid request body data results in:

Status Code: **422 Unprocessable Entity**

9. DELETE /employees/{emp_id}

Deletes an existing employee using the employee ID.

Path Parameter

Parameter	Type	Description
emp_id	Integer	ID of the employee to delete

Example Request

DELETE /employees/2

Successful Response

Status Code: **204 No Content**

No response body is returned.

Employee Not Found

Status Code: **404 Not Found**

```
json
{
  "detail": "Employee not found"
}
```

10. HTTP Status Codes

Status Code	Meaning	Description	Used In
200	OK	Request successful	GET, PUT
201	Created	Resource created	POST
204	No Content	Resource deleted successfully	DELETE

404	Not Found	Employee does not exist	GET, PUT, DELETE
422	Unprocessable Entity	Validation failed	POST, PUT, invalid ID